



고강현

Backend Developer

☎ 010-3754-9499

✉ rhrkdgus1020@naver.com

📍 경기도 남양주시 다산동



소개

앞으로 나아가며 성장하는 백엔드 개발자, Go 강현입니다. 기능을 구현하는 데서 끝나지 않고 어떤 사용자와 비즈니스 문제를 해결하는지 확인하며 개발하는 것을 중요하게 생각합니다. 초기 스타트업의 Founding Team으로 제품 기획부터 출시와 운영까지 참여했고, 누적 사용자 40만 명 규모의 서비스에서 DB 부하와 데이터 정합성 문제를 직접 해결했습니다. 운영 과정에서는 기획·디자인·마케팅 담당자와 문제를 정의하고, 반복 업무를 백오피스로 전환해 비개발자가 직접 서비스를 운영할 수 있도록 만들었습니다. Python과 TypeScript를 모두 활용하며 서비스 규모와 팀 상황에 맞는 기술을 선택하고, 변경이 실제 사용자 경험과 운영 효율에 어떤 영향을 주는지 끝까지 확인하는 개발자로 성장하고 있습니다.

기술

Backend	Python, FastAPI, TypeScript, Node.js, NestJS, REST API, BullMQ
Data	PostgreSQL, MongoDB, Redis, SQLAlchemy, Alembic
Frontend	Next.js, React
Infrastructure	AWS EC2·S3·Lambda, Docker, Docker Compose, GitHub Actions, Caddy, Sentry
Testing	pytest, pytest-asyncio, FastAPI TestClient

학력 및 수상

국민대학교 소프트웨어학과 특기자 전형	2019 - 2026.02 · 졸업
ICT 창의성 대회 은상	2018.08
국민대학교 알고리즘 대회 장려상	2018.08

경력

2025.07 - 2025.09	Grapic 근무 형태 파트타임 직무 풀스택 개발자 커뮤니티 데이터 수집 서버, 외부 콘텐츠 포스팅 연동과 정산·회원 권한·포스팅 관리 어드민 개발
2025.02 - 2025.07	Hashscape 근무 형태 계약직 직무 풀스택 개발자 React Native 기반 Solthumb 멀티체인 거래 서비스의 백엔드 아키텍처, 거래·자산 API와 앱 화면 개발
2024.07 - 2025.02	Hashscape 근무 형태 정규직 직무 Founding Team · 풀스택 개발자 Salt Hub Telegram 미니앱의 기획·설계부터 프론트엔드·백엔드 개발과 운영까지 참여해 사용자 40만 명 이상 확보
2023.07 - 2024.06	뉴웹커뮤니케이션 근무 형태 프리랜서 직무 자동화·웹 서버 개발자 콘텐츠 분석·생성·게시 과정을 연결한 마케팅 자동화 시스템과 변호사 플랫폼 MVP 개발

2023.03 - 2023.06	Inspien 근무 형태 인턴 직무 백엔드 개발자 분산된 B2B 서비스와 DB 통합, 내부 관리 시스템 및 사용자 메일 발송 기능 개발
경력 외 활동	
2026.01 - 2026.03	Nexters · Logit 활동 형태 IT 연합 동아리 직무 BE 리드 · 서버 개발자 Python·FastAPI 기반 서비스의 비동기 API, PostgreSQL·Redis 연동, 테스트와 팀 운영 어드민 개발
2022.06 - 2022.12	소프트웨어 마에스트로 13기 활동 형태 연수생 직무 팀장 · 서버/AI 개발자 NLP 기반 음악 자동 태깅 모델과 음악 태그 정보를 탐색·활용하는 애플리케이션 개발
Grapic	
파트타임 · 풀스택 개발자	2025.07 - 2025.09
커뮤니티 콘텐츠 수집·포스팅 자동화	
기술 스택	TypeScript, Node.js, Docker
본인의 역할	콘텐츠 수집 서버 개발
문제 정의	운영자가 여러 커뮤니티를 순회하며 콘텐츠를 찾고 이미지·본문을 서비스에 다시 입력해, 포스팅 한 건을 등록할 때마다 동일한 수작업이 반복됐습니다.
실행	커뮤니티별 페이지 구조를 분석해 제목·본문·이미지를 추출하는 수집 서버를 만들었습니다. 운영자가 URL만 입력하면 수집 결과를 기존 서비스의 포스팅 형식으로 변환해 등록하도록 API를 연동하고 Docker로 배포했습니다.
성과	운영 입력을 URL 1개로 줄이고, 콘텐츠 수집과 서비스 등록을 하나의 흐름으로 연결했습니다.
정산·회원·포스팅 운영 어드민 개발	
기술 스택	TypeScript, Node.js, React
본인의 역할	운영 어드민 풀스택 개발
문제 정의	정산 내역 확인, 회원 권한 변경과 포스팅 상태 수정이 DB 직접 작업에 의존해 운영 요청이 발생할 때마다 개발 업무가 중단됐습니다.
실행	업무별 조회·수정 범위를 구분한 관리 API를 만들고, 정산 조회와 회원 권한·포스팅 상태 변경을 운영자가 직접 수행하는 어드민 화면으로 연결했습니다.
성과	정산·회원 권한·포스팅 상태 등 3개 운영 영역을 개발자의 DB 직접 수정 없이 관리할 수 있게 했습니다.
Hashscape · Salt Hub	
정규직 Founding Team · 풀스택 개발자	2024.07 - 2025.02
메인 거래 서비스 출시 전 시장 반응과 초기 사용자를 확보하기 위해 만든 Telegram 미니앱입니다. 포인트 파밍, 퀘스트, 친구 초대, 흔들기 게임, 토큰 드롭 이벤트를 제공했습니다.	
담당 파트	Founding Team 풀스택 개발자로 인증·사용자·포인트·게임·초대·퀘스트·드롭 API와 웹 화면을 개발하고, 외부 협업 이벤트 조율과 운영 어드민 구축을 담당했습니다.
성과	누적 사용자 40만 명 이상과 MAU 2,000~3,000명을 기록해 시장 반응과 후속 서비스의 초기 사용자 기반을 검증했습니다.

Telegram 미니앱 제품 개발

기술 스택	TypeScript, NestJS, Next.js, MongoDB, Telegram Bot API, AWS
본인의 역할	미니앱 프론트엔드·백엔드 개발
문제 정의	후속 거래 서비스를 바로 개발하면 사용자 수요를 확인하지 못한 채 개발 비용을 투입할 위험이 있었고, 출시 시 활용할 초기 사용자 기반도 없었습니다.
실행	설치 없이 접근할 수 있는 Telegram 미니앱을 선택하고 인증·사용자·포인트·퀘스트·초대·게임 API와 웹 화면을 개발했습니다. 토큰 드롭과 협업 이벤트를 반복 운영할 수 있도록 참여 조건과 보상 흐름도 제품에 포함했습니다.
성과	누적 사용자 40만 명 이상과 MAU 2,000~3,000명을 기록해 후속 거래 서비스의 시장성과 초기 사용자 기반을 검증했습니다.

게임 점수 처리 부하 개선

기술 스택	NestJS, Redis, MongoDB
본인의 역할	게임 세션 및 점수 저장 로직 개발
문제 정의	사용자의 흔들기 입력마다 점수를 MongoDB에 저장하면 짧은 게임 한 번에도 다수의 쓰기가 발생해, 동시 사용자가 늘수록 DB 부하와 비용이 급격히 증가했습니다.
실행	게임 시작 시 사용자·IP·만료 시간을 포함한 세션을 발급하고 플레이 중 점수는 Redis에 임시 누적했습니다. 종료 요청에서 세션과 누적 점수를 검증한 뒤 최종 점수만 MongoDB에 한 번 저장했습니다.
성과	게임 중 반복되던 DB 쓰기를 게임 종료 시 1회로 줄여 MongoDB 사용량과 운영 비용을 낮췄습니다.

가입 직후 사용자 조회 오류 해결

기술 스택	NestJS, MongoDB Atlas
본인의 역할	사용자 가입·조회 API 개발
문제 정의	가입 API는 성공했지만 바로 이어진 사용자 조회에서 간헐적으로 데이터가 없다고 응답해 신규 사용자가 첫 화면에 진입하지 못했습니다.
실행	가입 쓰기 직후 다른 복제 노드에서 조회될 때 발생하는 지연을 원인으로 확인했습니다. 가입 데이터가 다수 노드에 반영된 것을 확인한 뒤 성공 응답을 반환하도록 해당 저장 작업에 `writeConcern: majority`를 적용했습니다.
성과	가입과 첫 사용자 조회 사이의 복제 지연을 제어해 신규 가입 흐름에서 발생하던 간헐적 조회 실패를 제거했습니다.

마케팅 지표 조회·리포트 기능 개발

기술 스택	TypeScript, NestJS, Next.js, MongoDB
본인의 역할	마케팅 데이터 어드민 개발
문제 정의	초대 성과, 가입자 수와 게임 이용량이 DB에만 있어 캠페인 성과를 확인할 때마다 개발자가 쿼리를 작성하고 파일을 전달해야 했습니다.
실행	마케팅 기준에 맞춰 사용자별 초대 인원, 전체 가입자 수와 게임 이용량을 집계하는 API를 만들고 어드민 대시보드에 연결했습니다. 조회 조건을 유지한 채 외부 공유용 PDF로 생성하는 기능도 추가했습니다.
성과	초대·가입·게임 이용 등 3개 핵심 마케팅 지표를 실시간 조회하고 PDF로 바로 공유할 수 있게 했습니다.

이벤트·배너·알림 운영 기능 개발

기술 스택	NestJS, Next.js, MongoDB
본인의 역할	서비스 운영 어드민 개발
문제 정의	협업 이벤트 일정이나 배너 문구가 바뀔 때마다 개발자가 DB를 수정하거나 재배포해야 해, 긴급 변경과 주말 운영이 개발자 대응 여부에 좌우됐습니다.
실행	이벤트와 배너의 내용·노출 기간·활성 상태를 데이터로 관리하도록 분리하고, 운영자가 수정 즉시 서비스에 반영할 수 있는 화면과 API를 개발했습니다. 사용자 알림도 대상과 내용을 입력해 직접 발송하도록 연결했습니다.
성과	이벤트·배너·알림 등 3개 운영 기능을 배포 없이 비개발자가 직접 변경할 수 있게 했습니다.

Hashscape · Solthumb

계약직 · 풀스택 개발자

2025.02 - 2025.07

한국 사용자가 Solana·Hedera·BSC·Ethereum의 밈 토큰을 거래하고 자산·거래 내역·추천 보상을 관리하는 멀티체인 서비스입니다. React Native로 iOS·Android·Web을 함께 제공했습니다.

담당 파트	백엔드 전반과 아키텍처 의사결정을 주도하고 체인별 거래·송금·수신·보상 처리, 거래 이력·포지션·시세 서버, 푸시 알림과 React Native 화면 일부를 개발했습니다.
성과	Salt Hub의 사용자 성과와 Solthumb의 멀티체인 제품 로드맵을 바탕으로 Hedera 재단의 Solthumb 개발 그랜트 계약을 체결했습니다.

체인별 거래 데이터 분석·표준화

기술 스택	TypeScript, NestJS, MongoDB, BullMQ
본인의 역할	멀티체인 거래 데이터 파이프라인 개발
문제 정의	4개 네트워크가 서로 다른 거래 기록을 반환했고, 원본 데이터에는 매수·매도 여부와 실제 수량·수수료가 정리되어 있지 않아 사용자 거래 내역을 만들 수 없었습니다.
실행	각 체인의 거래 전후 자산 변화와 전송 기록을 분석해 매수·매도·송금·수신·보상 여부를 판별했습니다. 여기서 토큰 수량과 네트워크·서비스 수수료를 추출해 모든 체인이 함께 사용하는 거래 이력 형식으로 저장했습니다.
성과	Solana·Hedera·BSC·Ethereum 4개 네트워크의 거래를 하나의 이력 스키마와 API로 통합했습니다.

거래 데이터 파싱 작업 유실 방지

기술 스택	NestJS, Redis, BullMQ, MongoDB
본인의 역할	비동기 거래 처리 구조 설계·개발
문제 정의	거래는 네트워크에서 완료됐지만 외부 조회 지연이나 서버 종료로 후속 분석이 중단되면, 온체인에는 거래가 있고 서비스 DB에는 이력이 없는 불일치가 발생했습니다.
실행	거래 식별값과 사용자 정보를 BullMQ의 영속 작업으로 먼저 등록해 서버가 재시작돼도 처리 대상을 복구하도록 했습니다. 네트워크별 작업을 분리하고 조회 실패 시 최대 10회, 5초 간격으로 재시도했으며 최종 실패 작업도 별도로 남겼습니다.
성과	서버 종료 후에도 작업을 재개하고 외부 데이터 지연에 최대 10회 대응해 온체인 거래의 오프체인 이력 누락 가능성을 낮췄습니다.

체인별 토큰 시세 수집 서버 개발

기술 스택	NestJS, Redis, BullMQ, MongoDB, GeckoTerminal·SaucerSwap API
본인의 역할	토큰 데이터 서버 설계·개발
문제 정의	네트워크마다 가격 제공처와 응답 형식이 달라 하나의 자산 화면에서 비교할 수 없었고, 여러 서버가 같은 수집 작업을 실행하면 외부 API 호출과 DB 갱신이 중복됐습니다.
실행	가격 수집 서버를 서비스 API와 분리하고 네트워크별 BullMQ 작업을 구성했습니다. Redis로 작업 실행 상태를 관리해 중복 수집을 막고 가격·원화 환산가·24시간 거래량·고가·저가·등락률을 같은 형식으로 저장했습니다.
성과	3개 네트워크의 가격·거래량·등락률을 하나의 API 형식으로 제공하고, 네트워크별 수집 작업을 동시에 1개만 실행하도록 제한했습니다.

온체인 거래와 보유 자산 데이터 정합성 확보

기술 스택	NestJS, MongoDB, Decimal.js
본인의 역할	사용자 포지션 계산 로직 개발
문제 정의	거래 분석 도중 서버가 종료되거나 같은 거래가 재처리되면 보유 수량이 누락·중복될 수 있었습니다. 또한 네트워크 수수료와 송금까지 반영하지 않으면 온체인 잔액과 서비스의 보유 자산이 달라졌습니다.
실행	거래 식별값과 사용자 ID를 복합 기준으로 이력을 upsert해 같은 거래의 중복 반영을 막고, 신규 수신 거래만 포지션에 반영했습니다. 성공한 온체인 거래를 기준으로 매수·매도·송금과 네트워크 수수료를 보유 수량에 반영하고 가중 평균 매수가를 다시 계산했습니다.
성과	4개 네트워크의 거래를 중복 없이 포지션에 반영하고, 수수료가 포함된 보유 수량·평균 매수가·평가 금액·수익률을 제공했습니다.

Nexters · Logit

IT 연합 동아리 · BE 리드 / 서버 개발자

2026.01 - 2026.03

취업 준비생이 자신의 경험을 구조화해 저장하고, 지원 회사·직무·자기소개서 문항에 맞는 경험을 추천받아 자기소개서를 작성하는 서비스입니다.

담당 파트	BE 리드로 백엔드 아키텍처와 API 규칙을 정하고 인증·경험·지원 프로젝트·문항·채팅 서버, RAG 추천 로직, MCP 서버, 운영 어드민과 dev/prod 배포 환경을 개발했습니다.
성과	Python·FastAPI 기반 비동기 API와 59개 자동화 테스트를 구축하고, 경험 추천·MCP·dev/prod 통합 운영 어드민을 제품에 적용했습니다.

RAG 도입과 경험 추천 성능 개선

기술 스택	Python, FastAPI, PostgreSQL, Qdrant, OpenAI API
본인의 역할	RAG 경험 추천 로직 개발
문제 정의	초기 추천은 자기소개서 문항과 단어가 비슷한 경험을 우선해, 지원 직무와 관련이 낮거나 회사 맥락에 맞지 않는 경험이 상위에 노출됐습니다.
실행	문항·직무·회사 정보를 각각 임베딩하고 문항 60%, 직무 25%, 회사 15%로 점수를 결합했습니다. 여기에 경험 태그의 Jaccard 유사도와 역량 카테고리 일치도를 보정값으로 추가했습니다.
성과	문항·직무·회사 3개 맥락과 태그·역량 2개 보정 기준을 함께 사용해 직무와 회사에 맞는 경험을 우선 추천하도록 개선했습니다.

비동기 API와 자동화 테스트 기반 구축

기술 스택	Python, FastAPI, SQLAlchemy, PostgreSQL, Alembic, pytest, Sentry
본인의 역할	백엔드 공통 구조 및 테스트 환경 구축
문제 정의	인증·사용자·프로젝트·경험·채팅 기능이 같은 DB와 외부 AI 서비스에 연결돼 있어, 한 기능의 변경이 다른 사용자 흐름을 깨뜨려도 배포 전에 확인하기 어려웠습니다.
실행	SQLAlchemy 비동기 세션과 asyncpg로 API의 DB 접근 방식을 통일하고 Alembic으로 스키마 변경을 관리했습니다. 테스트 DB 의존성 주입과 외부 API mock을 구성해 인증·사용자·경험·리포트 API를 독립적으로 검증했으며, Sentry와 요청 로깅 미들웨어로 배포 후 오류를 추적했습니다.
성과	4개 테스트 모듈에 59개 자동화 테스트를 구축해 주요 사용자 흐름의 회귀를 배포 전에 확인할 수 있게 했습니다.

취업 준비생의 비용 문제를 해결한 MCP 기능 도입

기술 스택	Python, FastMCP, PostgreSQL, Qdrant
본인의 역할	MCP 기능 제안 및 서버 설계·개발
문제 정의	사용자 인터뷰에서 취업 준비생은 비용 부담이 크고 이미 ChatGPT·Claude를 사용하고 있어, Logit의 AI 기능을 위해 새로운 구독료를 지불하거나 사용 도구를 바꿀 가능성이 낮았습니다.
실행	비개발자 팀원에게 MCP의 동작 방식과 사용자 흐름을 시연해 제품 방향을 합의했습니다. 사용자가 기존 AI에서 Logit 데이터를 활용하도록 경험 검색·저장, 문항별 경험 선택과 지원 프로젝트 조회 등 10개 도구를 구현하고 사용자별 데이터 접근을 분리했습니다.
성과	MCP 도구 10개를 제공해 사용자가 추가 AI 구독 없이 기존 AI에서 자신의 경험을 검색하고 자기소개서 작성까지 이어갈 수 있게 했습니다.

비개발자도 사용할 수 있는 운영 어드민 개발

기술 스택	Next.js, Prisma, PostgreSQL, Qdrant, TanStack Query, NextAuth, Docker, Caddy
본인의 역할	운영 어드민 설계·개발·배포
문제 정의	시연 중 가입·채팅·경험 데이터에 문제가 생겨도 비개발자는 상태를 확인할 수 없었고, dev와 production 데이터가 분리돼 개발자가 각 DB와 벡터 저장소를 직접 조회해야 했습니다.
실행	dev와 production 연결을 한 화면에서 전환하고 사용자·프로젝트·문항·경험·채팅을 조회·수정·삭제하는 어드민을 배포했습니다. PostgreSQL과 Qdrant 데이터를 결합해 가입자·활성 사용자·경험·채팅 사용량을 대시보드로 제공했습니다.
성과	dev/prod 2개 환경과 사용자·프로젝트·문항·경험·채팅 5개 운영 영역을 하나의 어드민에서 관리할 수 있게 했습니다.